



Become a Solution Architect

Enhance your Technical and Behavioural Skills

What's the
next thing
you're looking
forward to?



AI AGENTS ON AMAZON EKS

WORKSHOPS THAT DRIVE OUTCOMES



WEEK 3

LIVE ●

Week 4 - Agenda

Topic	Duration	Presenter
Technical Topic Building AI Agents on Amazon EKS	50 Mins	Ashish & Parna
Behavioural Topic Designing Workshops that Drive Outcomes	30 Mins	Jeff & Nisha
Workshop Walkthrough Building production ready AI Agents on Amazon EKS	30 Mins	Anmol

Pre Launch Q&A Session

BeSA Pre-Launch

«« How to get most out of BeSA sessions? »»



Schedule & Timings

- Plan ahead. Mark your calendar. Be on time.



Badges & Certificates

- Earn, showcase and grow your cloud skills.



How to Register for AWS Workshops (Hands-on)

- Step-by-step guide to secure your spot.

19 Aug
1 PM GMT

 **JOIN THE LIVE SESSION** >>



Additional Learning Resources



Whizlabs is providing the following free resources to all BeSA Cohort 10 participants (access is valid until 7th October):

1. Self-paced video course: Generative AI for AWS Cloud Engineers
2. Hands-on labs:
 - a. Database Management Using an AI Agent with n8n and RDS
 - b. GenAI-Powered Document Q&A Bot with LangChain

- Please reach out to Whizlabs directly at support@whizlabs.com



Building AI Agents on Amazon EKS



Fundamentals

Why Containers for AI Applications?

- AI applications can have complex and rapidly changing software dependencies. Containers package them into a **consistent, portable deployment unit**



Environment consistency

Locks in exact CUDA/library versions, avoiding “works on my machine” issues.



GPU resource isolation

Enables safe multi-job GPU sharing (via NVIDIA toolkit, MIG, etc.).



Reproducibility

Same code + data + environment = repeatable experiments and audits.



Easy scaling for inference

Autoscale, rolling updates, and load balancing for model serving.

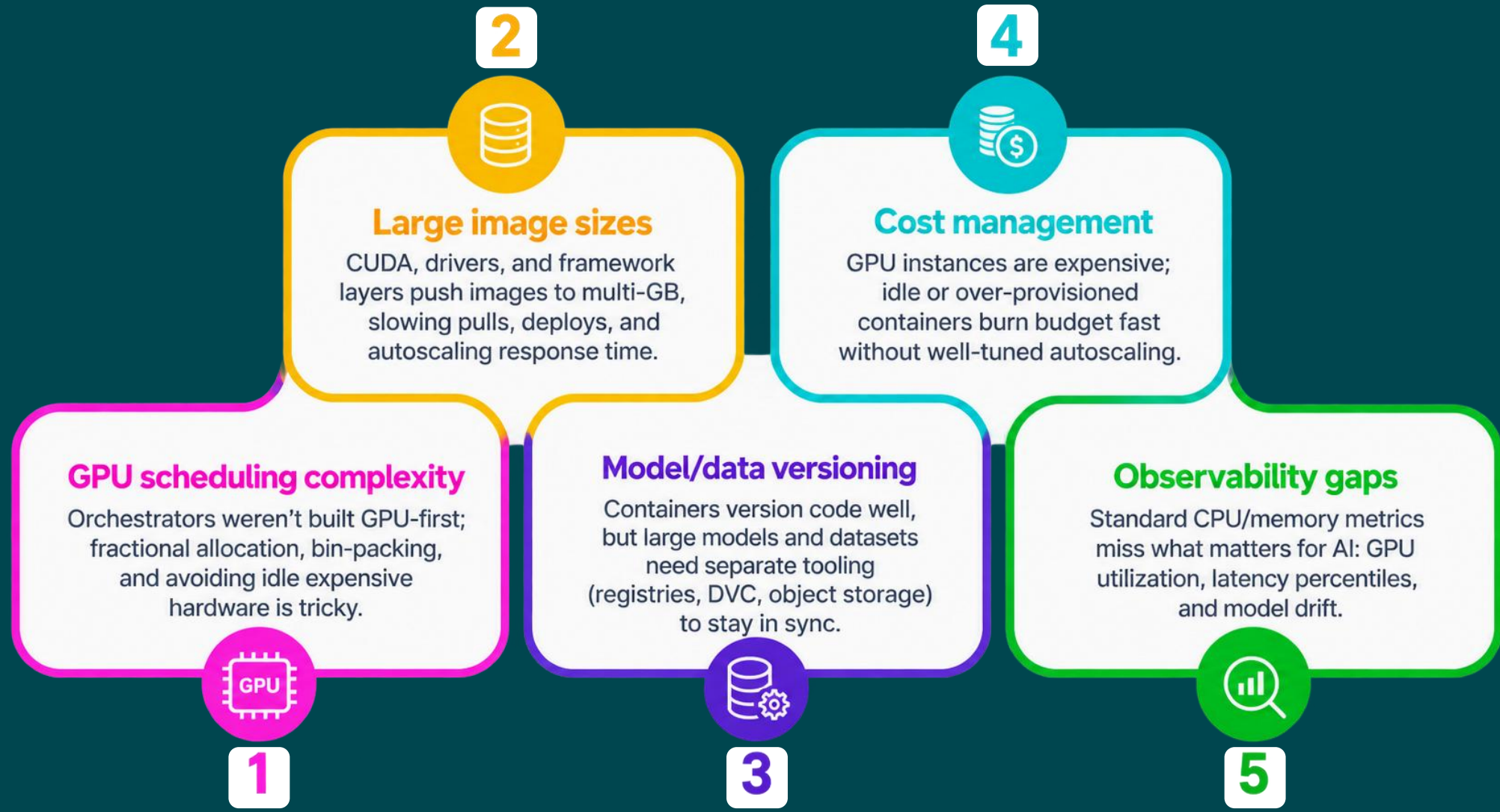


Portability

Move workloads across laptop, on-prem, or any cloud with minimal rework.

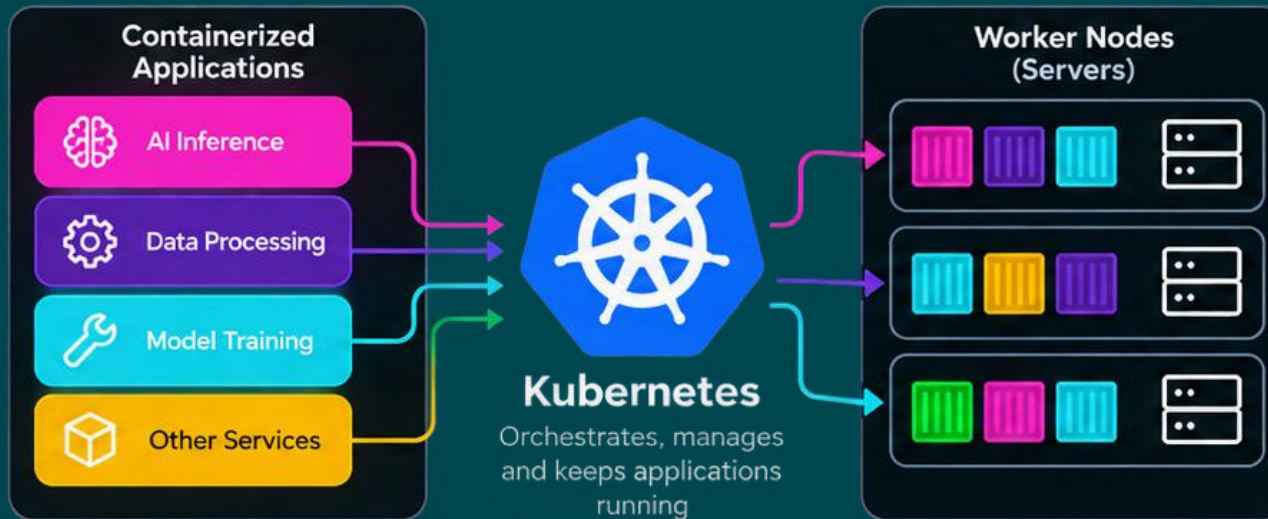
Managing Containers in Production is Challenging

- Running a container is easy. Running containerized AI applications reliably in production is much harder.



What is Kubernetes?

- Kubernetes is an open-source platform for orchestrating containerized applications.



Simple Mental Model



You define **WHAT** you want
"Run 3 instances of my
AI application"



Kubernetes figures out **HOW**

Schedule

Run

Monitor

Recover

Scale

Key Capabilities of Kubernetes

	Capability	What It Does
	Deploy	Places containerized applications across available machines.
	Schedule	Decides where Pods should run based on resources and constraints.
	Scale	Increases or decreases application Instances based on demand.
	Self-Heal	Detects unhealthy containers and restarts or replaces them.
	Update	Rolls out new application versions safely.
	Load Balance	Distributes traffic across application Instances.

An Airport

Air Traffic Controller

Coordinates and manages what happens across the airport

Terminal / Gate Area

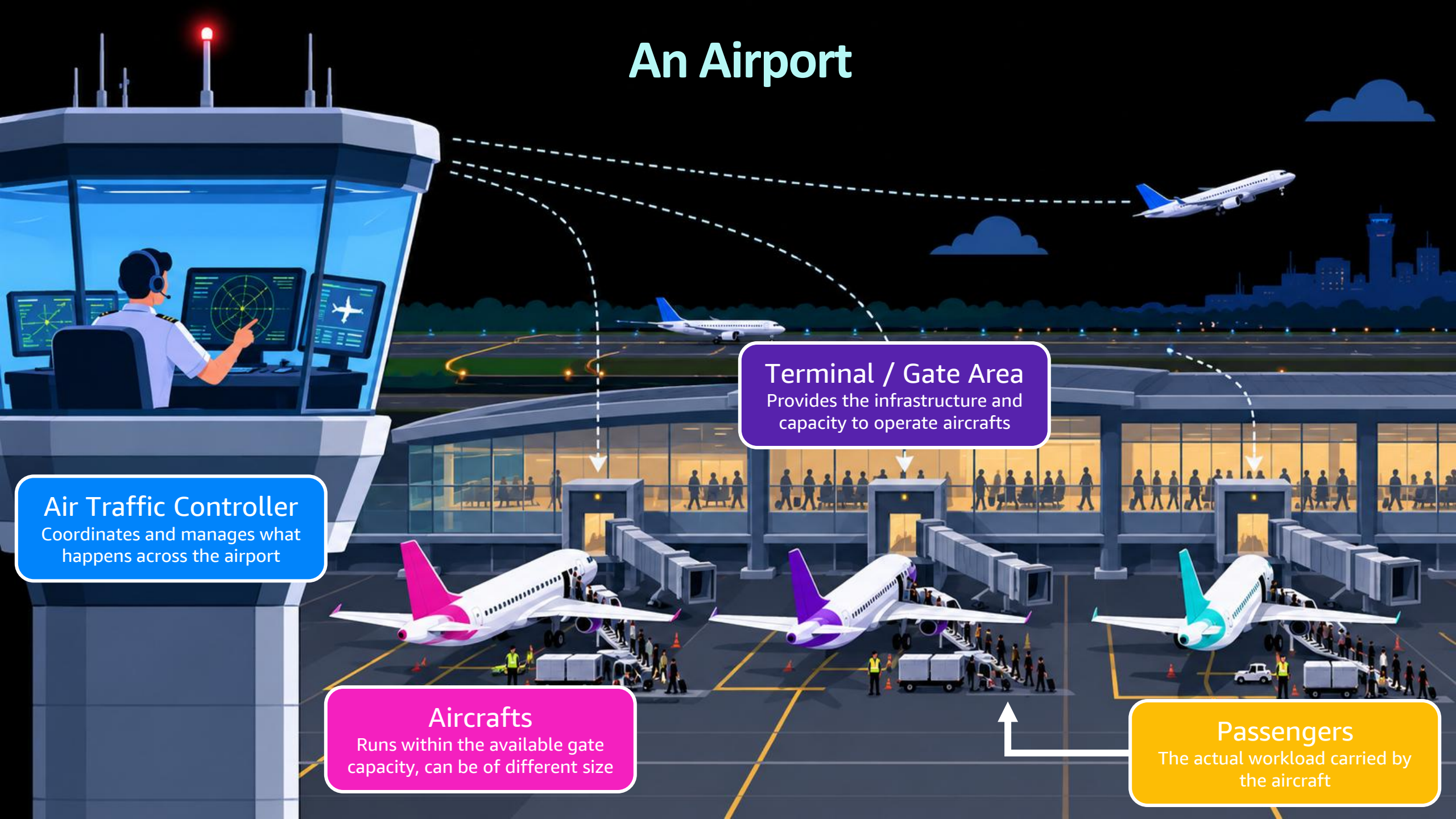
Provides the infrastructure and capacity to operate aircrafts

Aircrafts

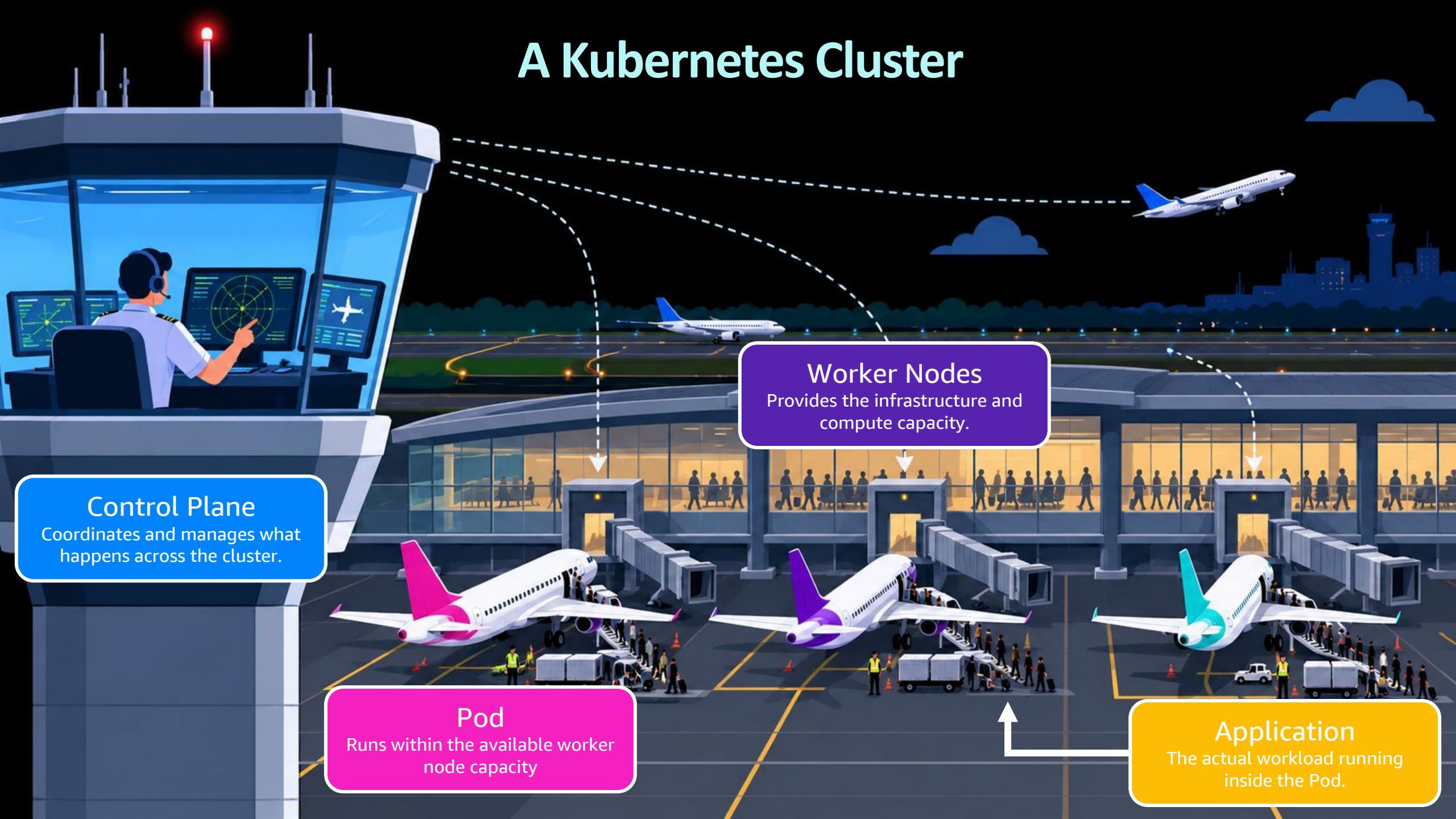
Runs within the available gate capacity, can be of different size

Passengers

The actual workload carried by the aircraft



A Kubernetes Cluster



Control Plane

Coordinates and manages what happens across the cluster.

Worker Nodes

Provides the infrastructure and compute capacity.

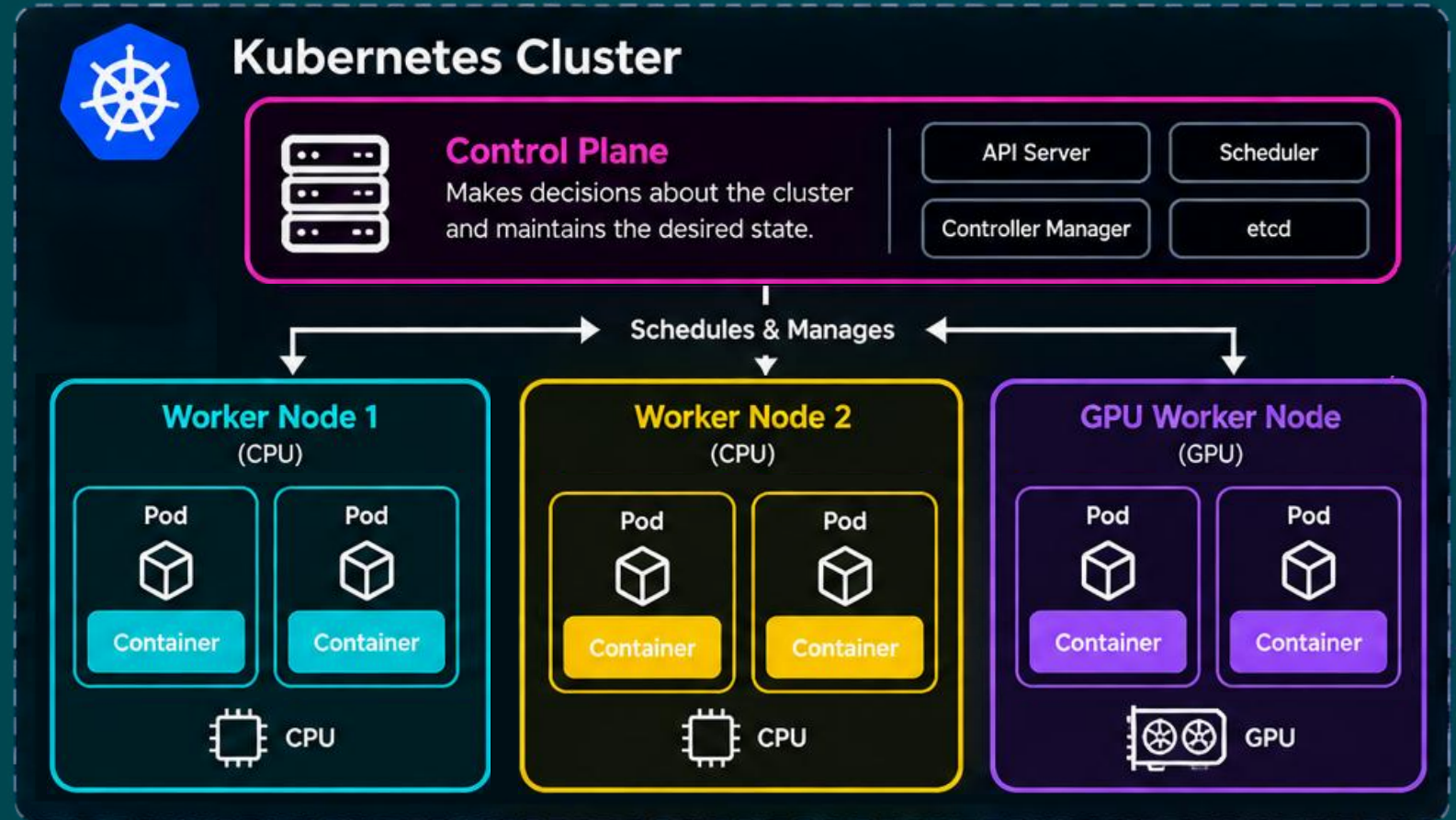
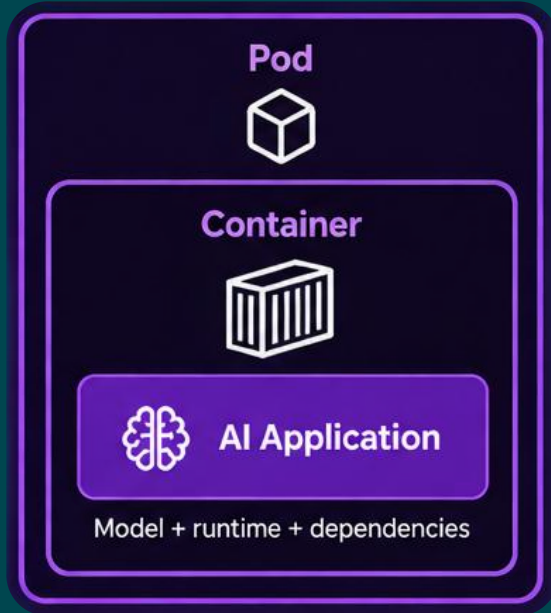
Pod

Runs within the available worker node capacity

Application

The actual workload running inside the Pod.

Kubernetes Building Block

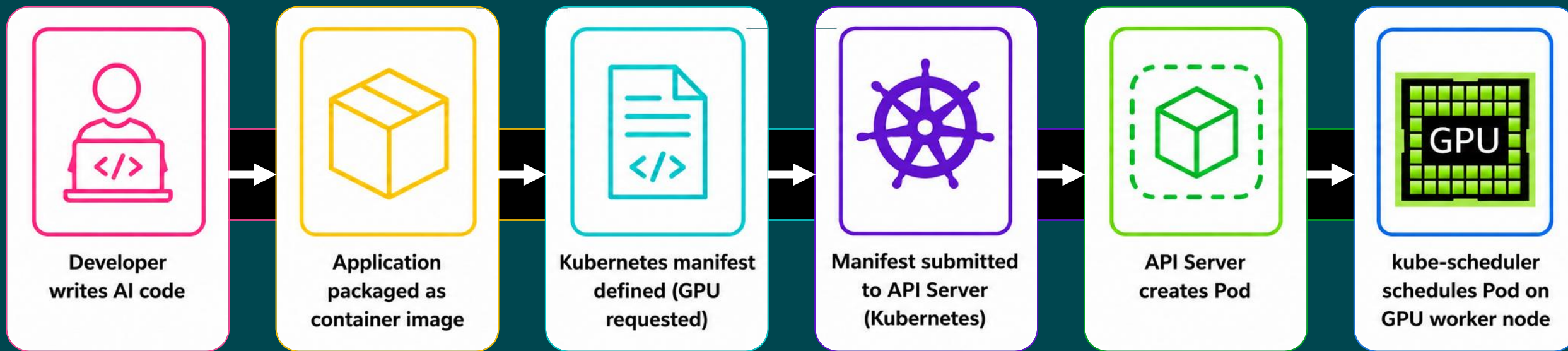


Key Takeaway

Control Plane manages → Worker Nodes provide compute → Pods run → Containers execute the application.

From AI Code to a Kubernetes Pod

- Application code may be GPU-aware, but it does not choose the Kubernetes worker node - the manifest declares the GPU resource requirement.
- Kubernetes handles infrastructure placement and selects a suitable worker node that can satisfy the Pod's GPU request.



Does Every Pod Need an Entire GPU?

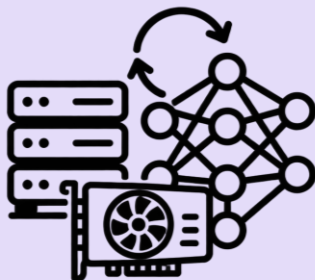
- Some workloads can make heavy use of an entire GPU, others may use only a fraction of its capacity.

Data Preparation / ETL



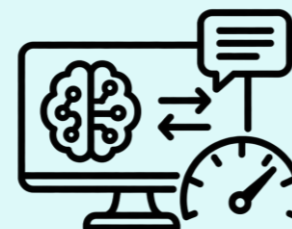
GPU Utilization:
Typically low.

Large-Model Training



GPU Utilization:
Typically high when optimized

Model Inference



GPU Utilization:
Depends, but often moderate

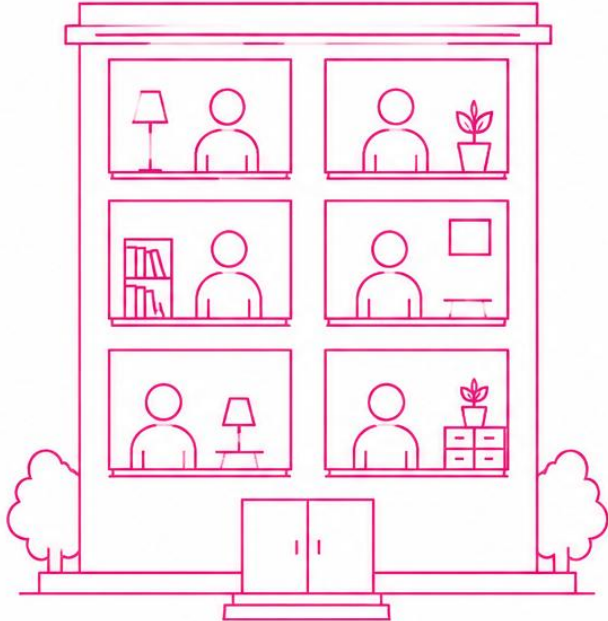
- Giving every workload exclusive access to a whole GPU can therefore leave resources underutilized.

Analogy – Resource Sharing

- Two ways to make better use of the same resource

Divide into Dedicated Spaces

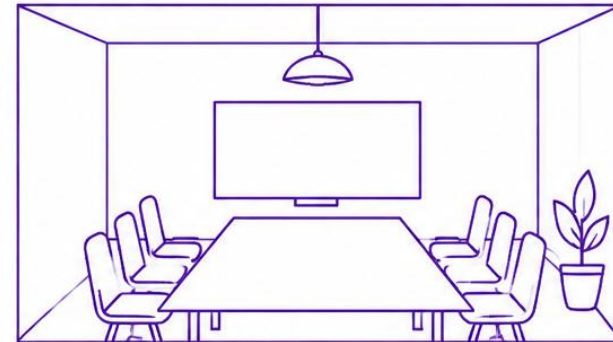
Analogy: Apartment Building



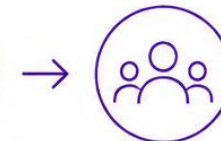
One building → separate apartments
Each tenant gets their own
dedicated space

Share the Same Space Over Time

Analogy: Meeting Room



Team A



Team B



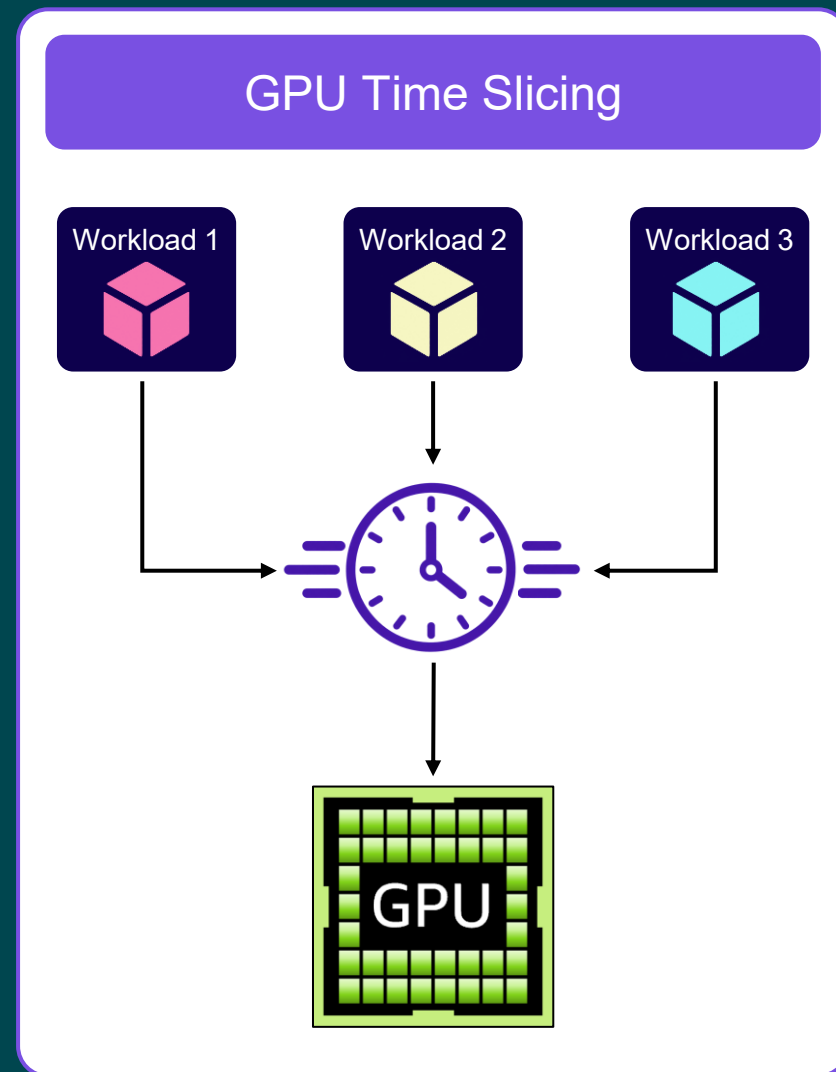
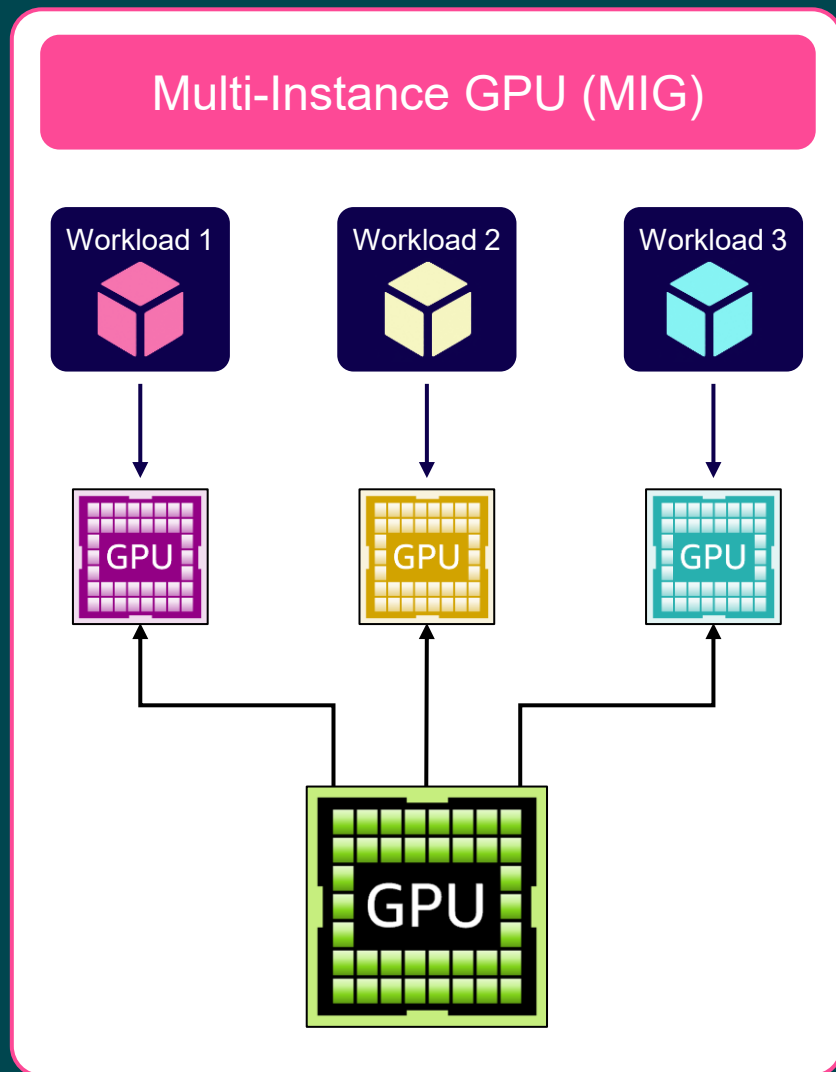
Team C



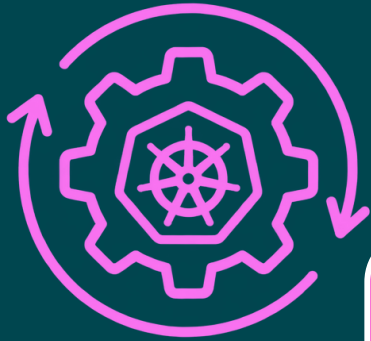
One meeting room → multiple teams
Each team takes a turn using
the same space

How Can Multiple Pods Share a GPU?

- Two ways to make better use of the same resource



Challenges in Managing Kubernetes



**Control plane
setup & upgrade**



**Cluster security
& compliance**

**Node
management
& scaling**



**Networking
& multi-AZ
resilience**



How Amazon EKS Helps?

EKS runs and manages the control plane (multi-AZ, auto-patched, SLA-backed), so teams just consume the API without operating etcd or API servers.

**Control plane
setup & upgrade**

**Cluster security
& compliance**

EKS integrates with IAM, supports encryption via KMS, and is certified for standards like SOC, HIPAA, and PCI out of the box.

Managed Node Groups, Karpenter, and Fargate automate node provisioning, patching, and scaling — including GPU node pools for AI workloads.

**Node
management
& scaling**

**Networking
& multi-AZ
resilience**

EKS integrates natively with VPC CNI, AWS Load Balancer Controller, and multi-AZ node groups, simplifying resilient networking out of the box.

Why Amazon EKS for AI Workloads?

- EKS combines Kubernetes with AWS infrastructure and AI/ML capabilities to deploy, scale, and optimize AI workloads.

	Flexible Compute	Run AI workloads across CPUs, NVIDIA GPUs, and AWS AI accelerators.
	Intelligent Scaling	Scale Pods and provision compute as workload demand changes.
	GPU Optimization	Schedule GPU resources and support GPU-sharing approaches for better utilization.
	Infrastructure Control	Tune compute, networking, storage, OS, and GPU configurations for workload requirements.
	Open Ecosystem	use Kubernetes APIs and integrate with open-source AI/ML frameworks and tooling.
	AWS Integration	Integrate workloads with AWS networking, IAM, storage, monitoring, and other AWS services.

What AI Workloads Can EKS Support?

- EKS is a flexible platform for a wide range of AI/ML use cases.

AI workloads **EKS can support**

EKS is a flexible platform for a wide range of AI/ML use cases.



Training

Train models
at scale



Fine-tuning

Customize
pre-trained
models



Inference

Serve models
in production



RAG

Retrieval
Augmented
Generation



Agentic AI

Build and run
AI agents

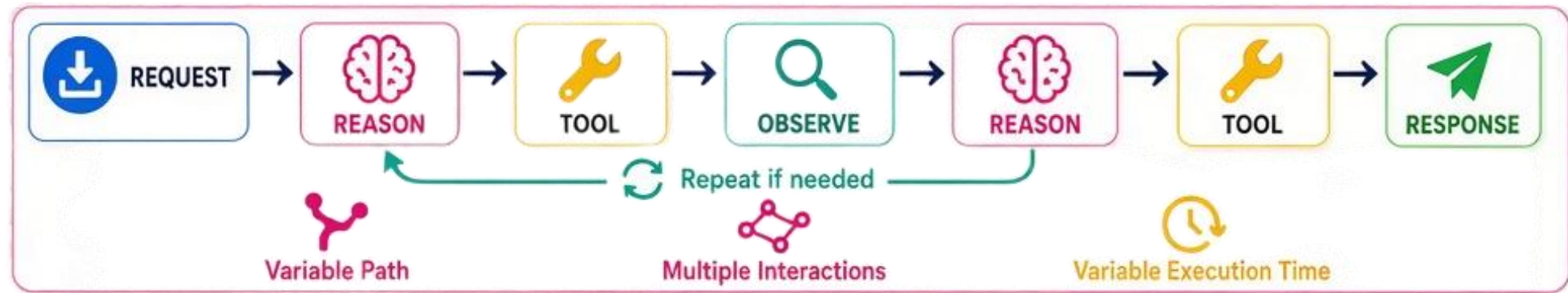
Why AI Agent Workloads are Different?

Traditional Application



- Agent workloads can be dynamic and long-running.

Agentic Application



Agent demand $\neq$ just HTTP request volume

Concurrency, active tasks, queues, latency, and custom metrics may become important scaling signals.



When Should You Choose EKS for Agentic AI?



EKS Is a Strong Fit

When you need Kubernetes-level control



- 1 Existing Kubernetes Platform**
reuse platform skills, tooling and operations.



- 2 Infrastructure Control**
fine-grained compute, networking and workload configuration.



- 3 Mixed Workloads**
run agents, tools, APIs and model workloads together.



- 4 Custom Scaling**
scale using signals that reflect actual demand.



- 5 Portability & Ecosystem**
Kubernetes APIs and cloud-native ecosystem.



CONTROL + FLEXIBILITY



SA DECISION



Do I need Kubernetes?



Consider a Simpler Option

When Kubernetes capabilities are not required.



- 1 Simple Agent**
few components and straightforward execution.



- 2 Minimal Infrastructure Control**
no specialized cluster configuration.



- 3 Lower Operational Overhead**
prefer less infrastructure to operate.



- 4 Event-Driven Workload**
execution fits managed or serverless compute.



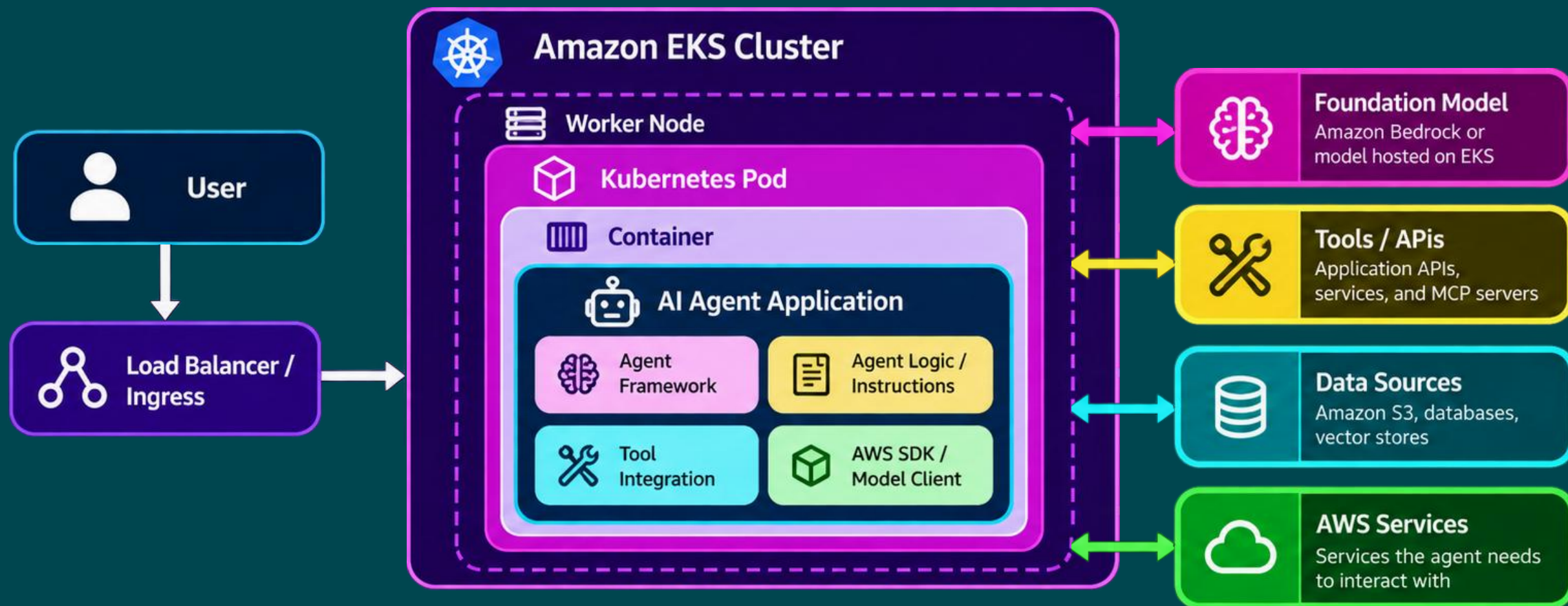
- 5 Faster Platform Simplicity**
avoid Kubernetes complexity when it adds little value.

A Common Architecture

- The agent can run on EKS and can connect to models, tools and data source.



How Do Users Access an AI Agent on EKS?



How to Scale Agentic Application on EKS?

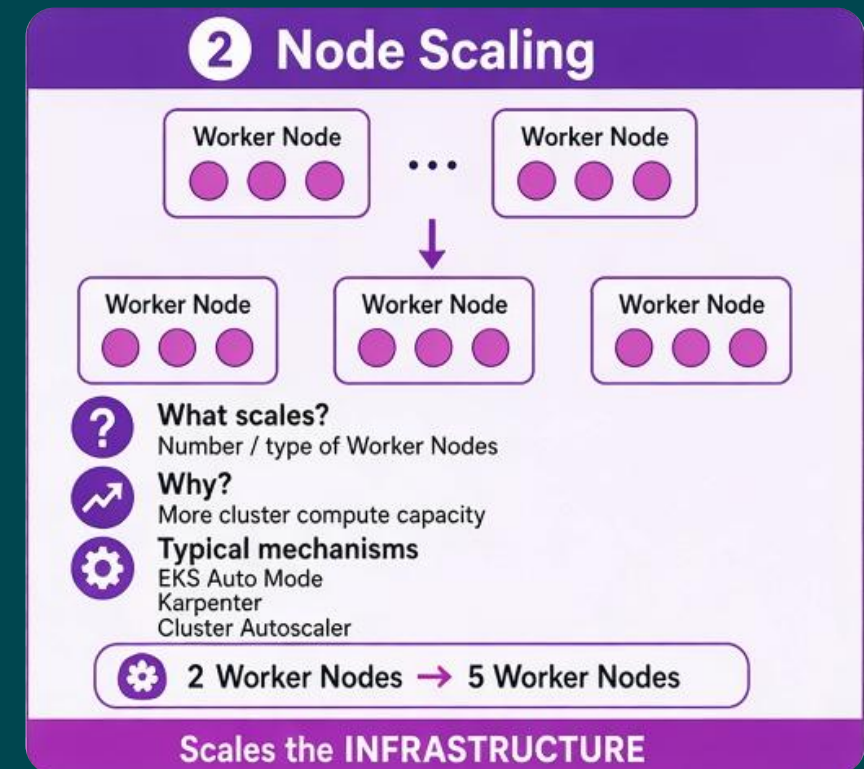
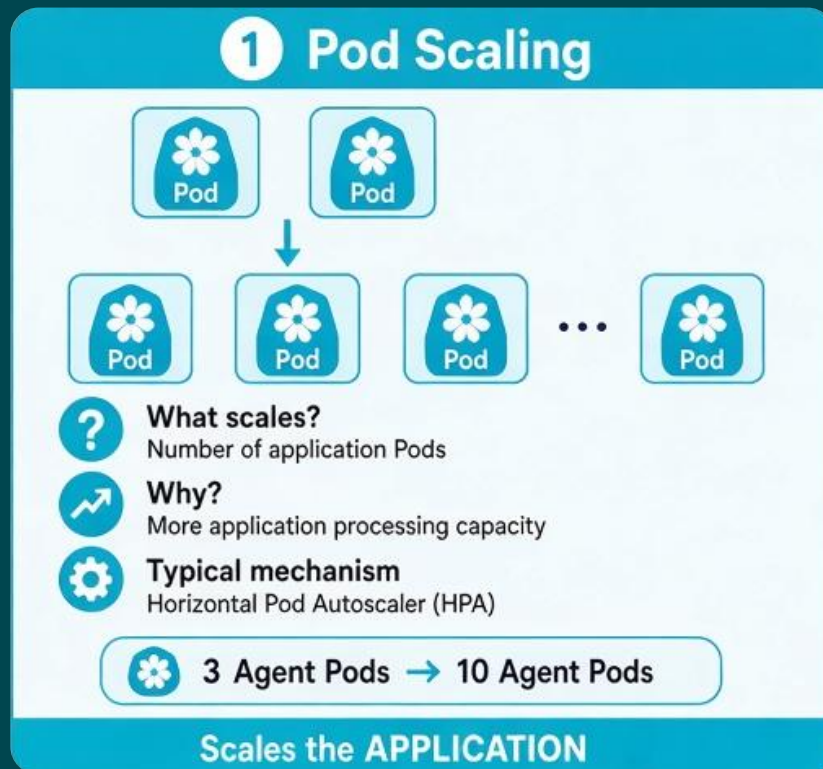
- Two ways to scale applications on EKS
 - Application Layer – Pod Scaling
 - Infrastructure Layer – Node Scaling



Worker Nodes Provide Capacity

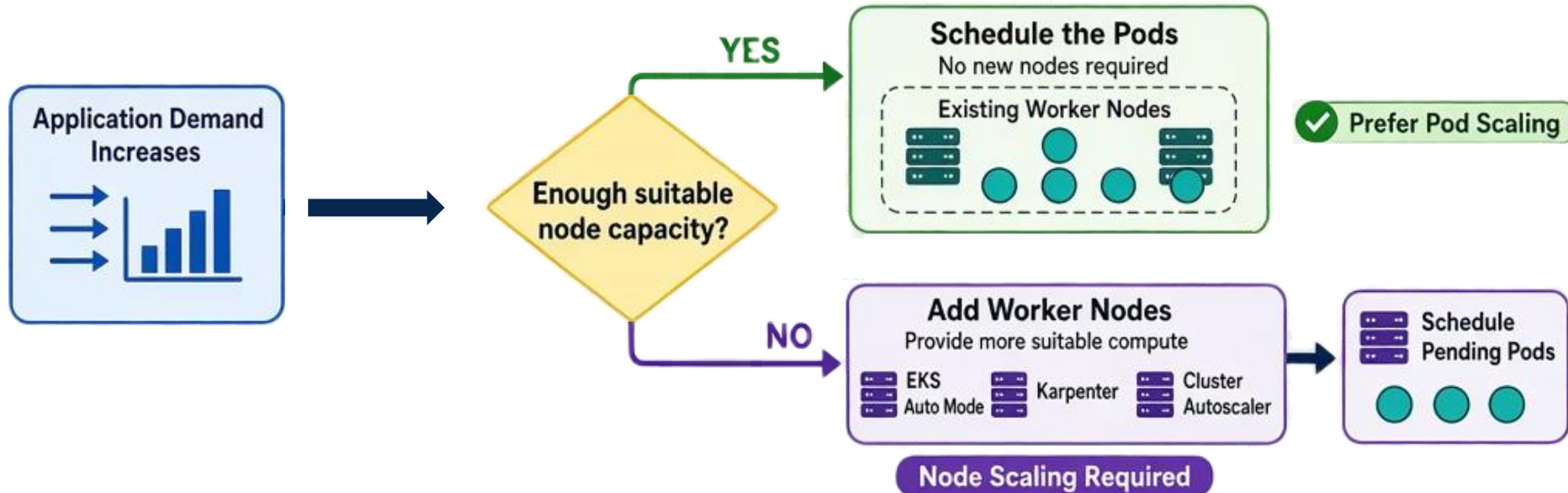


Pods Consume Capacity



When Should You Use Pod Scaling vs Node Scaling?

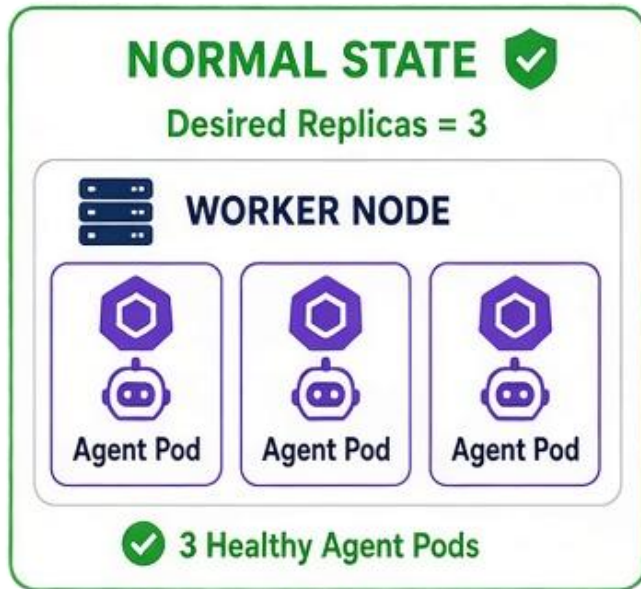
- Scale the workload first, add infrastructure when the workload needs more capacity



- Pod and Node scaling are complementary approaches, not competing.
- Dynamic or unpredictable workloads often need both layer of autoscaling.

Keeping AI Agents Available on EKS

- Kubernetes continuously works to maintain the desired state of application.
- Desired State
 - You declare what should be running, Kubernetes continuously works to maintain it.



— Pod Failure —



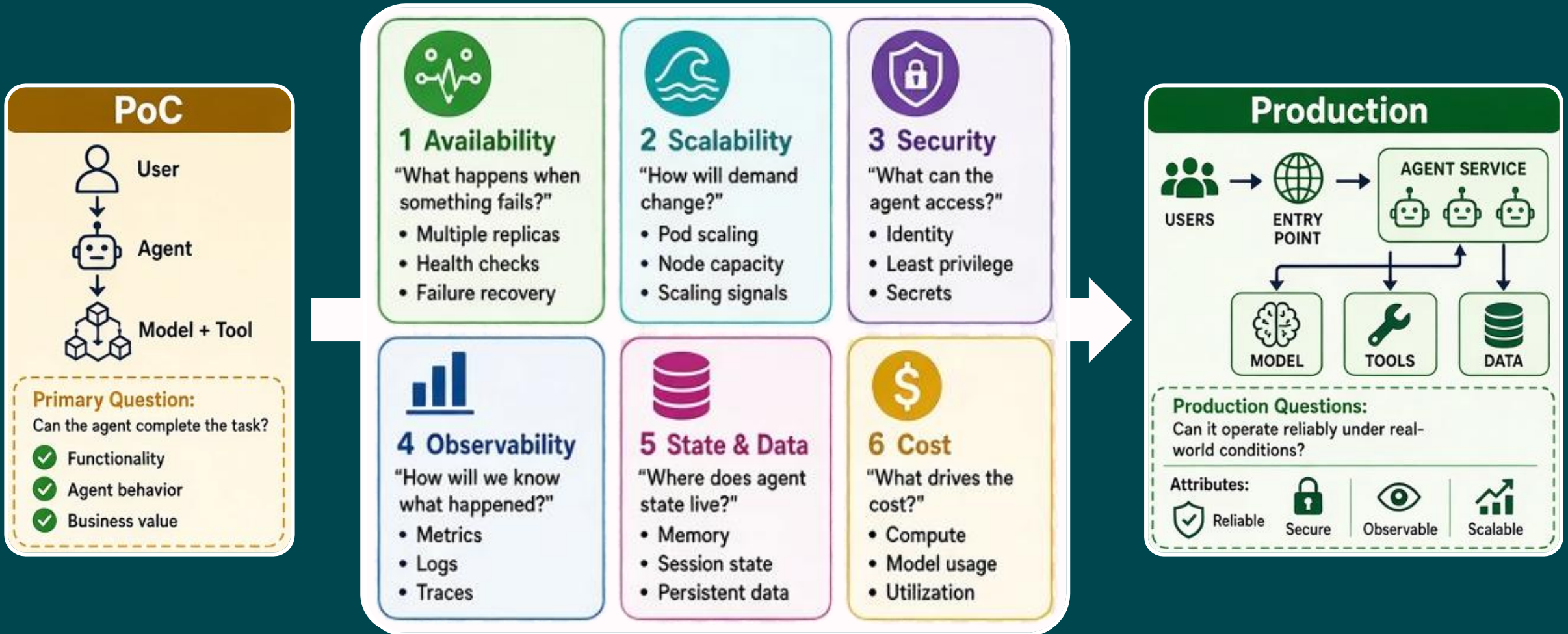
Kubernetes detects:

Actual = 2
Desired = 3



Agentic AI: PoC to Production on EKS

- A working agent proves the idea. Production requires architectural decisions.



The Journey



PoC to Production: Five Things to Remember

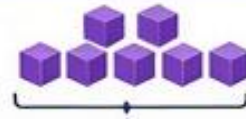
Choose EKS for a Reason



Use EKS when Kubernetes provides meaningful control, flexibility or operational value.

EKS is a choice — not an Agentic AI requirement.

Scale the Right Layer



PODS

Application Capacity



NODES

Infrastructure Capacity

Scale Pods for application demand and Nodes when more suitable compute capacity is required.

Design for Change



Expect Pods to fail, demand to change, and workloads to move.

Resilience must be designed.

Think Beyond the Agent



The agent is part of a distributed system involving models, tools, data and external services.

Production Is a System Property



SECURITY



OBSERVABILITY



STATE



COST

A working agent is only the beginning.

Validate the whole architecture.

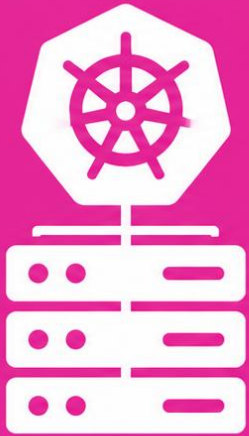


Agentic AI Patterns on EKS

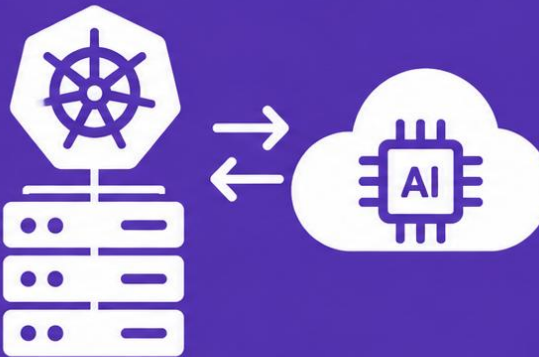
Agentic AI Patterns on EKS

- Each strategy involves distinct trade-offs between control, operational overhead, and time to production.

Self-Managed



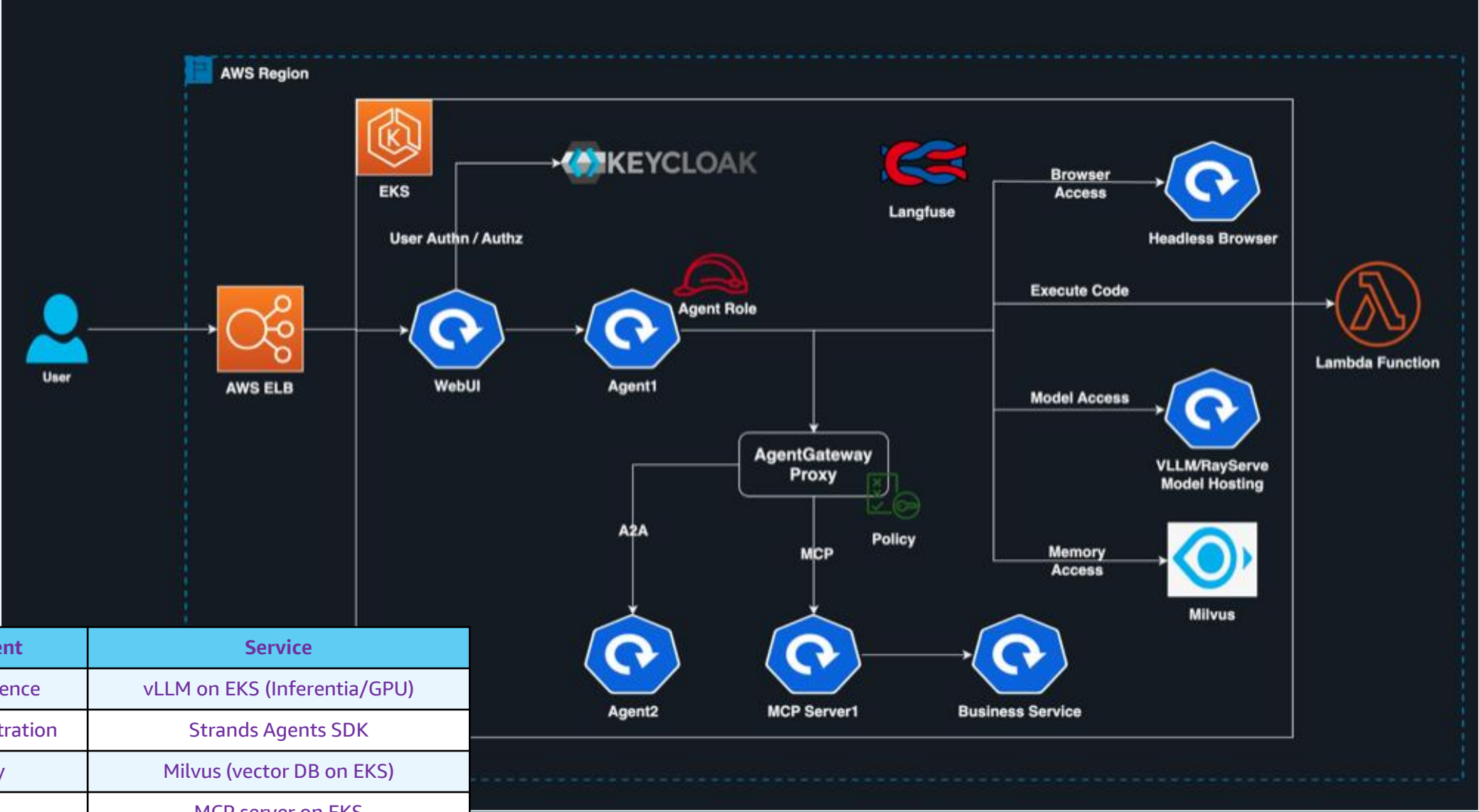
Integrated



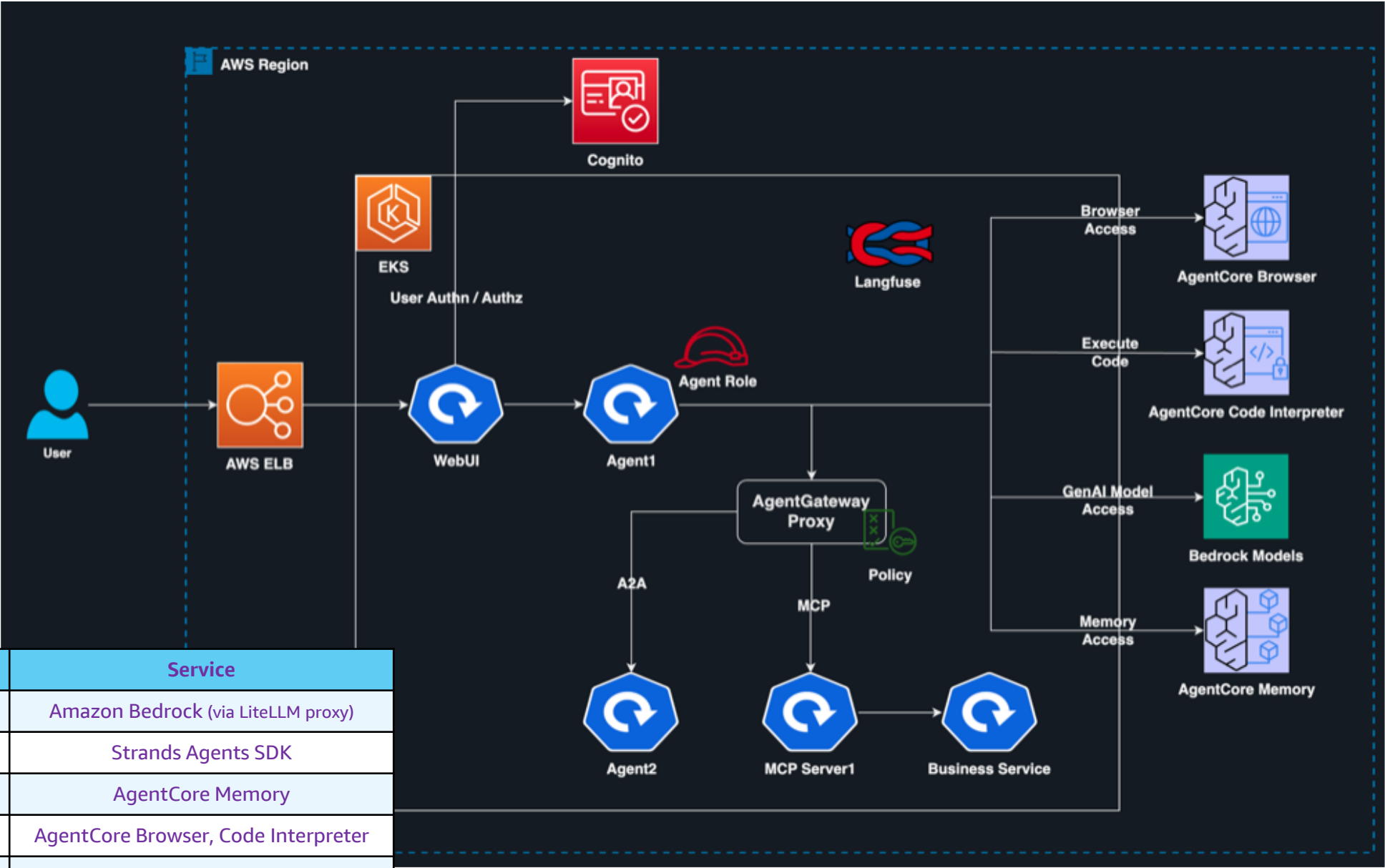
Fully Managed



Strategy 1: Self-Managed on Kubernetes

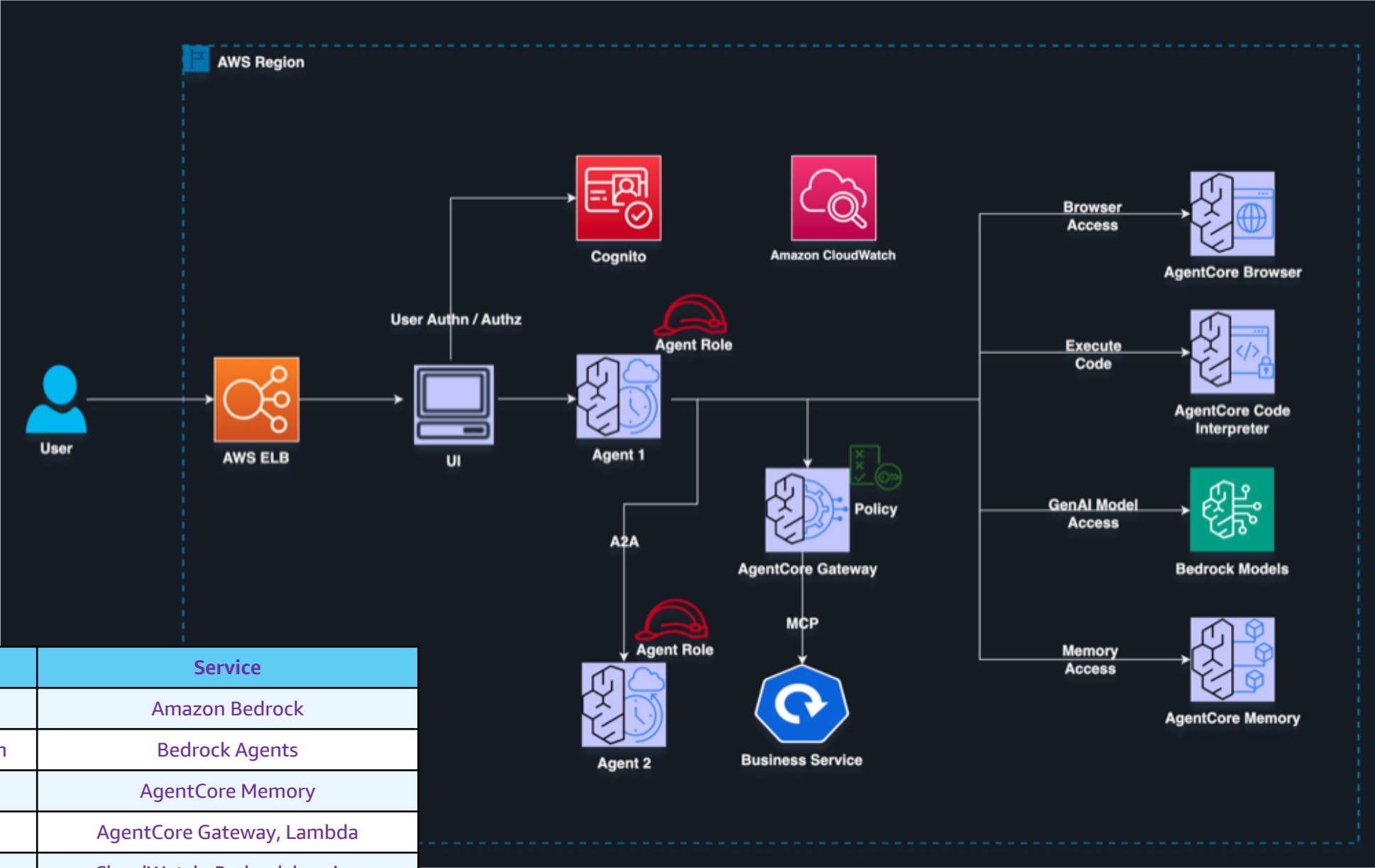


Strategy 2: Integrated (Self-Hosted Agents + Managed Agent Capabilities)



Component	Service
Model inference	Amazon Bedrock (via LiteLLM proxy)
Agent orchestration	Strands Agents SDK
Memory	AgentCore Memory
Tools	AgentCore Browser, Code Interpreter
Observability	Langfuse on EKS

Strategy 3: Fully Managed



Decision Framework

Question	Self-Managed	Integrated	Fully Managed
Need a specific open-source model?	Yes	No	No
Team has Kubernetes expertise?	Required	Required	Not required
Data must stay in your VPC?	Yes	Partial	Partial
Time to production matters most?	Slow	Middle	Best
Want cloud-portable architecture?	Yes	Partial	No
Complex multi-agent workflows?	Full control	Full control	Limited



Designing Workshops that Drive Outcomes



Hands-on Workshop

Walkthrough

Next Steps: Your Turn to Act

1



Complete the
Knowledge
Check

2



Practice the
Hands-on
Workshop

3



Be active and
Support
Your Peers

4



Share Your
Learning on
Social Media